

Coding Assignment 1: The Bag ADT as a Linked Chain

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

August 17, 2026

Due: two weeks from issue. There is no automatic late penalty — if you need more time, ask before the deadline and an extension will be granted (see the course policy in the syllabus).

1.1 Why this problem

Week 1 introduced the course’s central abstraction — the abstract data type (ADT) — with a running example: a **bag of integers** whose contract is exactly three operations, `insert(x)`, `member(x)`, and `size()` (Sedgewick & Wayne, §1.3, p.120; Karumanchi, §1.4, p.18). The lecture drew the two possible implementations — a dynamic array and a chain of linked nodes — but left the *code* for the linked version unwritten. This assignment is that code.

The point is not the code itself; it is the **pointer discipline** the lecture drilled: every `malloc` returns a pointer you must check against `NULL`; every `malloc` must eventually be matched by a `free`; and you never dereference `NULL`. A linked chain is the smallest program that forces you to practice all three at once.

1.2 Your task

Implement the Bag ADT in C as a **singly-linked chain of nodes on the heap**. The bag is represented by a pointer to its first node; the empty bag is `NULL`.

The contract is given to you in `bag.h` — *do not modify it*. Implement the four functions in `bag.c`:

```
Node *bag_insert(Node *head, int x); /* add x; return the new head */
int bag_member(const Node *head, int x); /* 1 if present, else 0 */
int bag_size(const Node *head); /* number of items */
void free_bag(Node *head); /* free every node (no-op on NULL) */
```

Insertion goes at the front. Allocate one new node with `malloc(sizeof(Node))`; if `malloc` returns `NULL`, leave the bag unchanged and return the original `head`. Otherwise set the node’s `data` to `x`, its `next` to the old `head`, and return the new node.

Membership and size walk the chain. `member` returns 1 the moment it finds a node whose `data` equals `x`, and 0 if it reaches the end; `size` counts nodes until it reaches `NULL`.

Freeing must visit every node. In `free_bag`, save `p->next` *before* calling `free(p)` — otherwise you read freed memory. Loop until `p` is `NULL`. Called on `NULL` (the empty bag), it must do nothing and not crash.

1.3 Constraints

- Representation must be a linked chain of heap nodes — no arrays, no global variables, no recursion.
- Correctness is graded against the contract, not against a particular coding style; but your code must compile with `gcc -Wall -Wextra` without errors.
- Memory discipline is part of the grade: every allocated node freed on `free_bag`; no use-after-free; no unchecked `malloc`.
- Complexity: `insert` should be $O(1)$ (front insertion needs no traversal); `member` and `size` are $O(n)$ by nature of a chain.

1.4 Worked examples (these are the visible tests)

Example 1 — the empty bag. Start with `head = NULL`: `size` is 0, `member(7)` is 0.

Example 2 — insert three values. After `insert(3)`, `insert(5)`, `insert(7)`: `size` is 3. `member(3)`, `member(5)`, and `member(7)` each return 1, while `member(9)` returns 0.

Example 3 — free and rebuild. After `free_bag`, the bag is empty again: `size` is 0. Insert 42: `member(42)` is 1 and `size` is 1.

1.5 What to submit

1. `bag.c` — your completed implementation (edit `starter_bag.c`).
2. `bag.h` — unchanged, but include it so the submission compiles standalone.

1.6 Getting started

1. Copy `starter_bag.c` to `bag.c` and fill in the four function bodies.
2. Sanity-check locally:

```
gcc -Wall -Wextra -o test_visible test_visible.c bag.c
./test_visible
```

All three worked examples should pass.

3. Submit `bag.c` and `bag.h`. The autograder compiles your `bag.c` against a *hidden* test suite and reports pass/fail. The hidden suite covers empty/single-element boundaries, front/middle/tail positions, duplicates, negatives, a 10,000-node chain, and reuse after `free`.